

Universidad de las Fuerzas Armadas – ESPE

Departamento de Ciencias de la Computación

Ingeniería en Tecnologías de la Información

Modelado UML — Diagrama de casos de uso, Diagrama Puente

Grupo 1

Integrantes

Eatan Joao Ayala Maldonado

Johan Mateo Catota Analuisa

Lenin Esteban Cuichan Males

Katerine Julisa Caizaluisa Guzman

Metodología de Desarrollo Software

30693

Ing. Jenny Ruiz

15 de Julio 2026

Contenido

1. Listado de Requisitos Funcionales “Huertos el Juank”	3
1.1. Tabla de Requisitos Funcionales por Sprint.....	4
2. Diagrama de clases	4
2.1. Código en Plant UML	4
3. Sprint 1.....	13
3.1. Diagrama de Casos de Uso REQ001 – REQ005	13
3.2. Puente Sprint 1	14
3.3. Diagrama Puente PlantUML Sprint 1	14
4. Sprint 2.....	15
4.1. Diagrama de Casos de Uso REQ006 -REQ009.....	15
4.2. Puente Sprint 2	17
4.3. Diagrama Puente PlantUML Sprint 2	18
5. Sprint 3.....	19
5.1. Diagrama de Casos de Uso REQ010 – REQ013	19
5.2. Puente Sprint 3	21
5.3. Diagrama Puente PlantUML Sprint 3	21

1. Listado de Requisitos Funcionales “Huertos el Juank”

Tabla 1 Requisitos Funcionales

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
[REQ001]	El sistema debe permitir navegar entre los diferentes módulos del aplicativo.	Administrador del Sistema	Navegar por el Sistema
[REQ002]	El sistema debe permitir registrar productos con información de nombre, precio, stock y disponibilidad.	Administrador del Sistema	Registrar Producto
[REQ003]	El sistema debe permitir modificar o eliminar productos registrados mediante un catálogo.	Administrador del Sistema	Gestionar Producto
[REQ004]	El sistema debe permitir registrar clientes con sus datos personales y dirección.	Administrador del Sistema	Registrar Cliente
[REQ005]	El sistema debe permitir actualizar o eliminar información de clientes registrados.	Administrador del Sistema	Gestionar Cliente
[REQ006]	El sistema debe permitir visualizar alertas de cumpleaños de clientes próximos a cumplir años.	Administrador del Sistema	Consultar Alerta de Cumpleaños
[REQ007]	El sistema debe permitir registrar clientes potenciales.	Administrador del Sistema	Registrar Cliente Potencial
[REQ008]	El sistema debe permitir modificar o eliminar clientes potenciales registrados.	Administrador del Sistema	Gestionar Cliente Potencial
[REQ009]	El sistema debe permitir visualizar, reactivar o eliminar clientes inactivos.	Administrador del Sistema	Gestionar Cliente Inactivo
[REQ010]	. El sistema debe permitir almacenar el catálogo de productos.	Administrador del Sistema	Catálogo de Productos

[REQ011]	El sistema debe permitir registrar ventas asociadas a clientes y productos	Administrador del Sistema	Registrar Venta
[REQ012]	El sistema debe permitir generar e imprimir notas de venta.	Administrador del Sistema	Generar nota de Venta
[REQ013]	El sistema debe permitir generar reportes en formato Excel.	Administrador del Sistema	Generar Reportes Excel

1.1. Tabla de Requisitos Funcionales por Sprint

Tabla 2 Requisitos Funcionales por Sprint

Sprint	Requisitos Funcionales
Sprint 1	• [REQ001] • [REQ002] • [REQ003] • [REQ004] • [REQ005]
Sprint 2	• [REQ006] • [REQ007] • [REQ008] • [REQ009]
Sprint 3	• [REQ010] • [REQ011] • [REQ012] • [REQ013]

2. Diagrama de clases

2.1. Código en Plant UML

@startuml

skinparam packageStyle rectangle

skinparam classAttributeIconSize 0

hide empty members

title Gestión Los Huertos - Diagrama UML (MVC)

```
'=====
' MODELOS
'=====
```

```
package "Model" {

class Cliente{

-id : String

-nombre : String

-apellido : String

-cedula : String

-telefono : String

-direccion : String

-estado : String

-fechaCumpleaños : LocalDate

+getId():String

+setId(id:String)

+getNombre():String

+setNombre(nombre:String)

+getCedula():String

+setCedula(cedula:String)

}

class ClientePotenciales{

-codigoClientePotencial:String

-nombreClientePotencial:String

-apellidoClientePotencial:String

-telefonoClientePotencial:String

}

class Producto{

-idProducto:int

-codigoProducto:String
```

```
-nombreProducto:String  
-tipoEmpaque:String  
-tamanoProducto:String  
-precioVenta:double  
-stockProducto:int  
-estadoProducto:String  
-rutaImagen:String  
}
```

```
class Venta{  
-idVenta:String  
-fecha:LocalDate  
-idCliente:String  
-total:double  
}
```

```
class DetalleVenta{  
-idVenta:String  
-codigoProducto:String  
-nombreProducto:String  
-cantidad:int  
-precioUnitario:double  
  
+getSubtotal():double  
}
```

```
}
```

```
'=====
```

```
' CONTROLLERS
```

```
'=====
```

```
package "Controller" {  
  
class ClienteRepository{  
+buscarPorId(id:String):Cliente  
+buscarPorCedula(cedula:String):Cliente  
+listarClientes():ArrayList  
+guardarCliente(cliente:Cliente):boolean  
+modificarCliente(cliente:Cliente):boolean  
+eliminarCliente(id:String):boolean  
}
```

```
class ClientePotencialRepository{  
+listar():ArrayList  
+guardar(cp):boolean  
+modificar(...):boolean  
+eliminar(...):boolean  
}
```

```
class ProductoRepository{  
+buscarPorCodigo(codigo):Producto  
+listarProductos():ArrayList  
+guardarProducto(p):boolean  
+modificarProducto(p):boolean  
+eliminarProducto(id):boolean  
}
```

```
class VentaRepository{  
+guardarVenta(v):boolean  
+listarVentas():ArrayList  
+buscarVenta(id):Venta
```

}

```
class DetalleVentaRepository{  
+guardarDetalle(d):boolean  
+listarDetalles():ArrayList  
+buscarPorVenta(id):ArrayList  
}
```

```
class ClienteService{  
-vista:Frm_Principal  
-dao:ClienteRepository
```

```
+cargarTabla()  
+cargarClientePotencial(cp)  
}
```

```
class ClientePotencialService{  
-vista:Frm_CliPotenciales  
-dao:ClientePotencialRepository  
}
```

```
class ProductoService{  
-vista:Frm_Principal  
-dao:ProductoRepository
```

```
+cargarTabla()  
}
```

```
class VentaService{  
-vista:Frm_Principal  
-ventaRepo:VentaRepository
```



```
-detalleRepo:DetalleVentaRepository  
-productoRepo:ProductoRepository  
-clienteRepo:ClienteRepository  
}
```

```
class ReporteService{  
-vista:Frm_Principal  
-ventaRepo:VentaRepository  
-detalleRepo:DetalleVentaRepository  
-clienteRepo:ClienteRepository  
}
```

```
class ClientelnactivoService  
  
}
```

```
'=====
```

```
' VISTAS
```

```
'=====
```

```
package "View" {
```

```
class Frm_Principal
```

```
class Frm_CliPotenciales
```

```
class Frm_CatalogoProductos
```

```
class Frm_ClientesInactivos
```

```
class Frm_PantallaCarga
```

```
class Frm_VistaPreviaPDF{
    -ventaActual:Venta
    -clienteActual:Cliente
    -detallesActuales:ArrayList<DetalleVenta>
```

```
+getBtnConfirmarImpresion():JButton
}
```

```
}
```

```
'=====
' RELACIONES MODELO
'=====
```

```
Venta *-- "1..*" DetalleVenta
Venta --> Cliente
DetalleVenta --> Producto
```

```
'=====
' REPOSITORIOS
'=====
```

```
ClienteRepository --> Cliente
ProductoRepository --> Producto
VentaRepository --> Venta
DetalleVentaRepository --> DetalleVenta
ClientePotencialRepository --> ClientePotenciales
```

```
'=====
' SERVICES
```

'=====

ClienteService --> ClienteRepository

ClienteService --> Frm_Principal

ProductoService --> ProductoRepository

ProductoService --> Frm_Principal

VentaService --> VentaRepository

VentaService --> DetalleVentaRepository

VentaService --> ProductoRepository

VentaService --> ClienteRepository

VentaService --> Frm_Principal

ReporteService --> VentaRepository

ReporteService --> DetalleVentaRepository

ReporteService --> ClienteRepository

ReporteService --> Frm_Principal

ClientePotencialService --> ClientePotencialRepository

ClientePotencialService --> Frm_CliPotenciales

ClientelInactivoService --> Frm_ClientesInactivos

'=====

' VISTAS

'=====

Frm_Principal ..> ClienteService

Frm_Principal ..> ProductoService

Frm_Principal ..> VentaService

Frm_Principal --> ReporteService

Frm_CatalogoProductos --> ProductoRepository

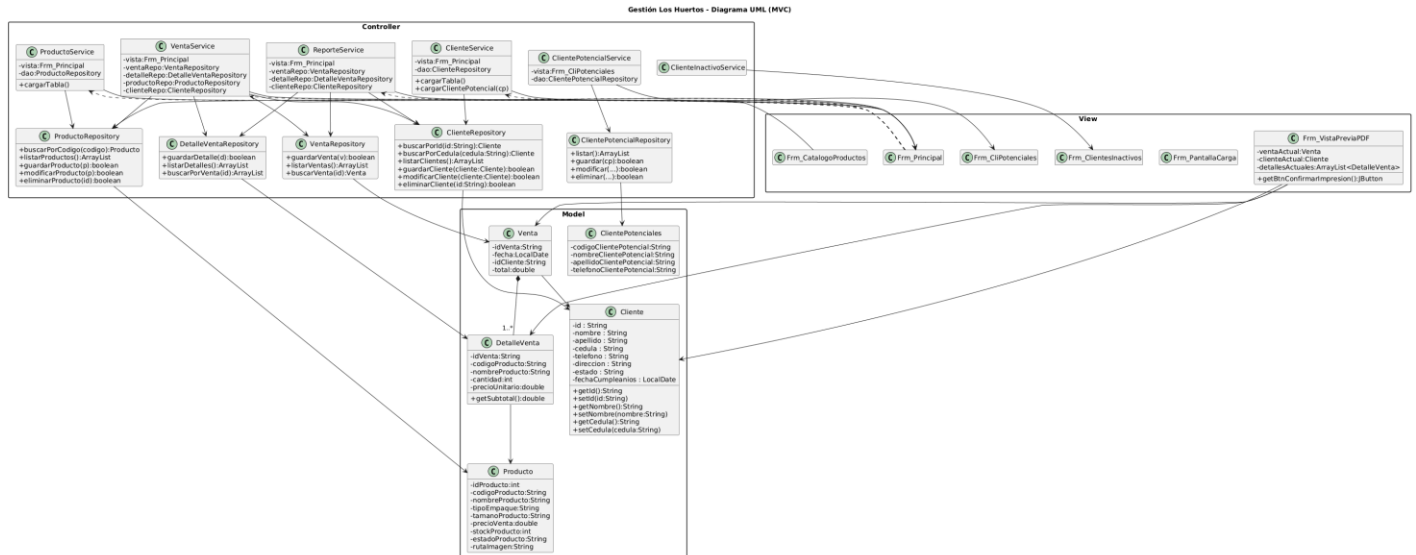
Frm_VistaPreviaPDF --> Venta

Frm_VistaPreviaPDF --> Cliente

Frm_VistaPreviaPDF --> DetalleVenta

@enduml

2.2. Diagrama de clases



3. Sprint 1

3.1. Diagrama de Casos de Uso REQ001 – REQ005

- **Código PlantUML**

@startuml Sprint1

left to right direction

skinparam backgroundColor white
skinparam shadowing true
skinparam roundcorner 20

skinparam actorStyle awesome

```
skinparam actor {  
    BackgroundColor #ed4519  
    BorderColor #BF9000  
    FontColor #000000  
    FontStyle bold  
}
```

```
skinparam rectangle {  
    BackgroundColor #ffffff  
    BorderColor #000000  
    FontColor #000000  
    FontStyle bold  
    RoundCorner 25  
}
```

```
skinparam usecase {  
    BackgroundColor #8fe4ed  
    BorderColor #000000  
    FontColor #000000  
    FontStyle bold  
}
```

skinparam ArrowColor #000000
skinparam ArrowThickness 2

actor "Administrador\n del Sistema" as Admin

rectangle "Sprint 1 - Navegación y Funciones Cliente/Producto" {

usecase "[REQ001] Navegar por el Sistema" as UC1
usecase "[REQ002] Registrar Producto" as UC2
usecase "[REQ003] Gestionar Producto" as UC3
usecase "[REQ004] Registrar Cliente" as UC4
usecase "[REQ005] Gestionar Cliente" as UC5

}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Admin --> UC4
Admin --> UC5

@enduml

- **Captura del Diagrama PlantUML Sprint 1**

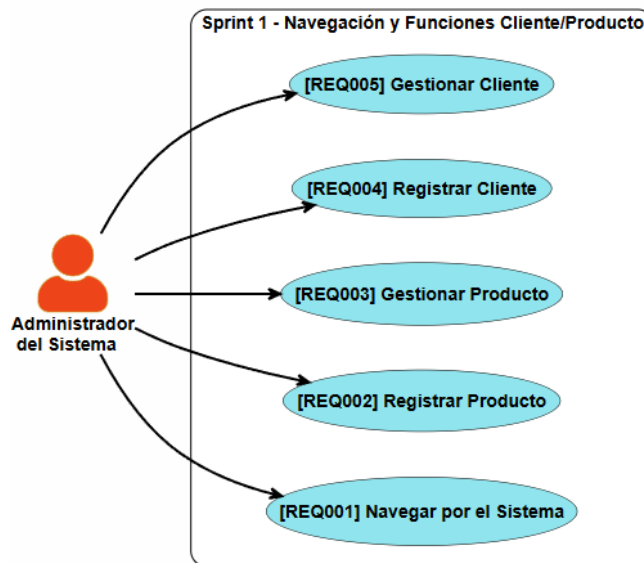


Ilustración 1 Diagrama 1 PlantUML

3.2. Puente Sprint 1

Tabla 3 Puente hacia Clases Sprint 1

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante
Frontera / Boundary	Representa la interacción entre el usuario y el sistema	Frm_Principal, Frm_PantallaCarga
Control / Control	Coordina la lógica de los casos de uso	ProductoService, ClienteService
Entidad / Entity	Representa la información almacenada del sistema	Producto, Cliente

3.3. Diagrama Puente PlantUML Sprint 1

- **Código PlantUML**
@startuml PuenteSprint1

left to right direction

skinparam backgroundColor white

skinparam shadowing false

actor "Administrador\n del Sistema" as Admin

boundary "Frm_Principal" as UI

control "ProductoService" as ProdSrv

control "ClienteService" as CliSrv

entity "Producto" as Producto

entity "Cliente" as Cliente

Admin --> UI : navegar()

UI --> ProdSrv :

guardarProducto()\nmodificarProducto()\neliminarProducto()\nbuscarProducto()

UI --> CliSrv :

guardarCliente()\nmodificarCliente()\neliminarCliente()\nbuscarCliente()

ProdSrv --> Producto : gestiona datos

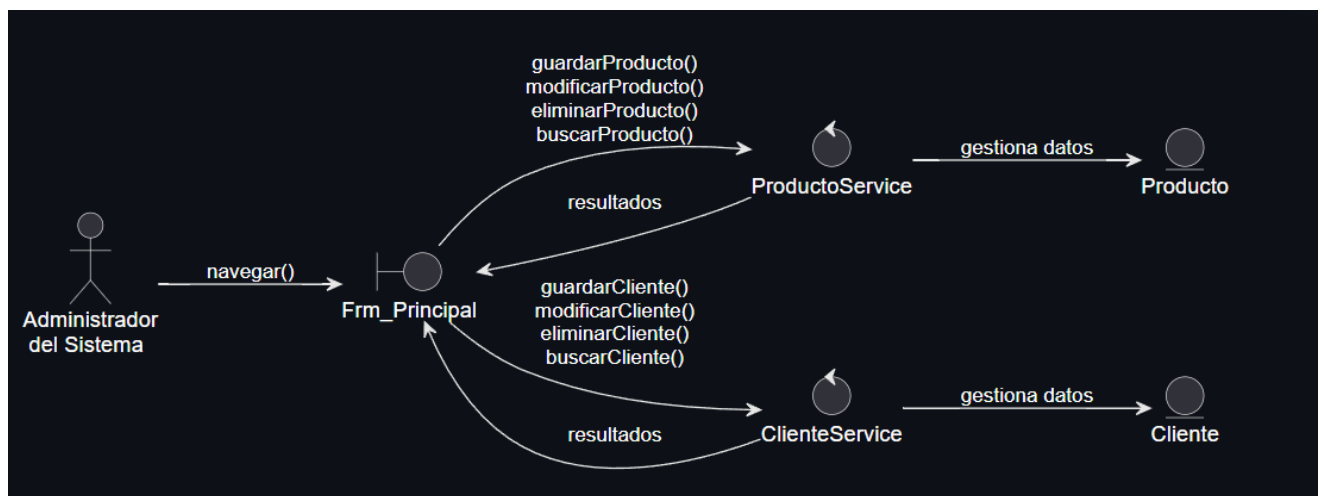
CliSrv --> Cliente : gestiona datos

ProdSrv --> UI : resultados

CliSrv --> UI : resultados

@enduml

- **Captura Diagrama Puente PlantUML Sprint 1**



4.1. Diagrama de Casos de Uso REQ006 -REQ009

- **Código PlantUML**

@startuml Sprint2

left to right direction

skinparam backgroundColor white

skinparam shadowing true

skinparam roundcorner 20

skinparam actorStyle awesome

skinparam actor {

 BackgroundColor #ed4519

 BorderColor #BF9000

 FontColor #000000

 FontStyle bold

}

skinparam rectangle {

 BackgroundColor #ffffff

 BorderColor #000000

 FontColor #000000

 FontStyle bold

 RoundCorner 25

}

skinparam usecase {

 BackgroundColor #78dbc0

 BorderColor #000000

 FontColor #000000

 FontStyle bold

}

skinparam ArrowColor #000000

skinparam ArrowThickness 2

actor "Administrador\n del Sistema" as Admin

rectangle "Sprint 2 - Gestión y Seguimiento de Clientes" {

usecase "[REQ006] Alerta de Cumpleaños" as UC6

usecase "[REQ007] Registrar Cliente Potencial" as UC7

usecase "[REQ008] Gestionar Cliente Potencial" as UC8

usecase "[REQ009] Gestionar Cliente Inactivo" as UC9

}

Admin --> UC6

Admin --> UC7

Admin --> UC8

Admin --> UC9

@enduml

- **Captura de Diagrama PlantUML Sprint 2**

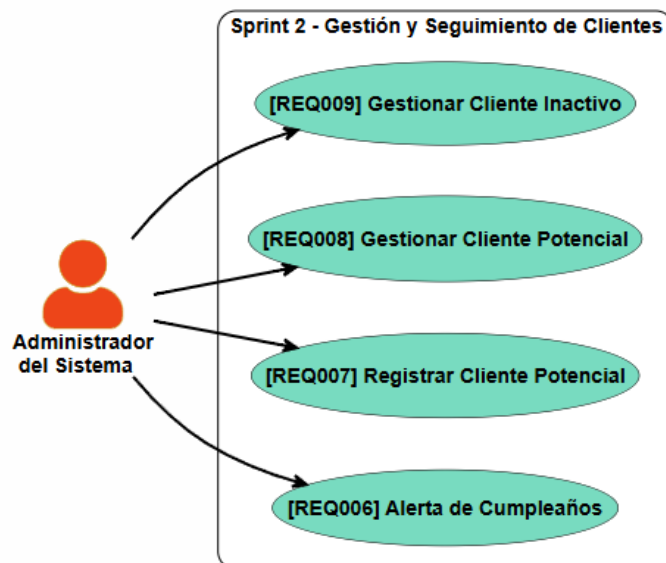


Ilustración 2 Diagrama 2 PlantUML

4.2. Puente Sprint 2

Tabla 4 Puente hacia Clases Sprint 2

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante
---------------	-----------------------	------------------------------

Frontera / Boundary	Representa la interacción entre el usuario y el sistema	Frm_Principal, Frm_CliPotenciales Frm_ClientesInactivos
Control / Control	Coordina la lógica de los casos de uso .	ClienteService, ClientePotencialService
Entidad / Entity	Representa la información almacenada del sistema	Cliente, ClientePotenciales

4.3. Diagrama Puente PlantUML Sprint 2

- *Codigo PlantUML*

@startuml PuenteSprint2

left to right direction

skinparam backgroundColor white

skinparam shadowing false

actor "Administrador\n del Sistema" as Admin

boundary "Frm_Principal" as MainUI

boundary "Frm_CliPotenciales" as PotUI

control "ClienteService" as CliSrv

control "ClientePotencialService" as PotSrv

entity "Cliente" as Cliente

entity "ClientePotenciales" as Potencial

Admin --> MainUI : cumpleaños()\ninactivos()

Admin --> PotUI : potenciales()

MainUI --> CliSrv : verificarCumpleañosHoy()

PotUI --> PotSrv : guardar()\nmodificar()\neliminar()

CliSrv --> Cliente

PotSrv --> Potencial

@enduml

- Captura Diagrama Puente PlantUML Sprint 2

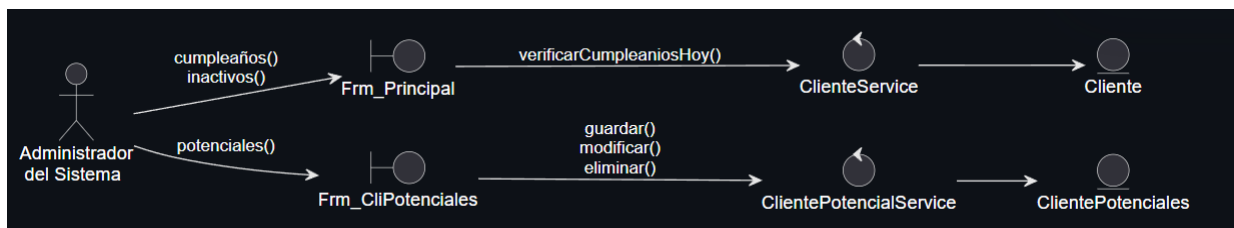


Ilustración 6 Puente PlantUML Sprint 2

5. Sprint 3

5.1. Diagrama de Casos de Uso REQ010 – REQ013

- *Código UML*

@startuml Sprint3

left to right direction

skinparam backgroundColor white

skinparam shadowing true

skinparam roundcorner 20

skinparam actorStyle awesome

skinparam actor {

BackgroundColor #ed4519

BorderColor #BF9000

FontColor #000000

FontStyle bold

}

skinparam rectangle {

 BackgroundColor #ffffff

 BorderColor #000000

 FontColor #000000

 FontStyle bold

 RoundCorner 25

}

skinparam usecase {

 BackgroundColor #788bdb

 BorderColor #000000

 FontColor #000000

 FontStyle bold

}

skinparam ArrowColor #000000

skinparam ArrowThickness 2

actor "Administrador\n del Sistema" as Admin

rectangle "Sprint 3 - Ventas y Reportes" {

 usecase "[REQ010] Catálogo de Productos" as UC10

 usecase "[REQ011] Registrar Venta" as UC11

 usecase "[REQ012] Generar Nota de Venta" as UC12

 usecase "[REQ013] Generar Reportes Excel" as UC13

}

Admin --> UC10

Admin --> UC11

Admin --> UC12

Admin --> UC13

@enduml

- **Captura del Diagrama de Casos de Uso Sprint 3**

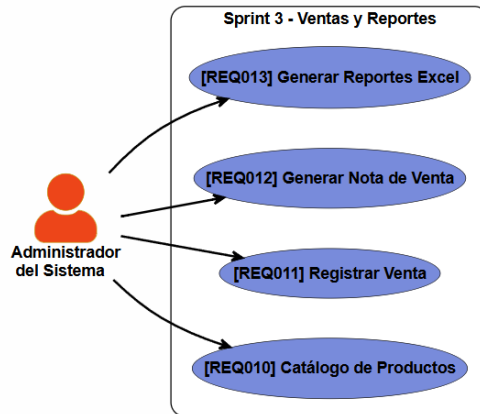


Ilustración 3 Diagrama 3 PlantUML

5.2. Puente Sprint 3

Tabla 5 Puente hacia Clases Sprint 3

Tipo de clase	Propósito en análisis	Ejemplo para CRUD Estudiante
Frontera / Boundary	Representa la interacción entre el usuario y el sistema	Frm_Principal, Frm_CatalogosProductos Frm_VistaPreviaPDF ProductoSelector
Control / Control	Coordina la lógica de los casos de uso	ReporteService VentaService .
Entidad / Entity	Representa la información almacenada del sistema	Venta DetalleVenta

5.3. Diagrama Puente PlantUML Sprint 3

- **Codigo PlantUML**
@startuml PuenteSprint3

left to right direction

actor "Administrador\n del Sistema" as Admin

boundary "Frm_Principal" as UI

boundary "Frm_CatalogoProductos" as Catalogo

control "VentaService" as VentaSrv

control "ReporteService" as ReporteSrv

entity "Producto" as Producto

entity "Venta" as Venta

entity "NotaVenta" as Nota

entity "Reporte" as Reporte

Admin --> UI : gestionar ventas\nreportes

UI --> Catalogo : abrir catálogo

Catalogo --> VentaSrv : seleccionar producto

UI --> VentaSrv : registrar venta

VentaSrv --> Producto : consulta productos

VentaSrv --> Venta : guarda venta

UI --> ReporteSrv : generar nota\nreportes

ReporteSrv --> Nota : genera nota

ReporteSrv --> Reporte : genera reporte

VentaSrv --> UI : resultado

ReporteSrv --> UI : resultado

@enduml

- **Captura Diagrama Puente PlantUML Sprint 3**

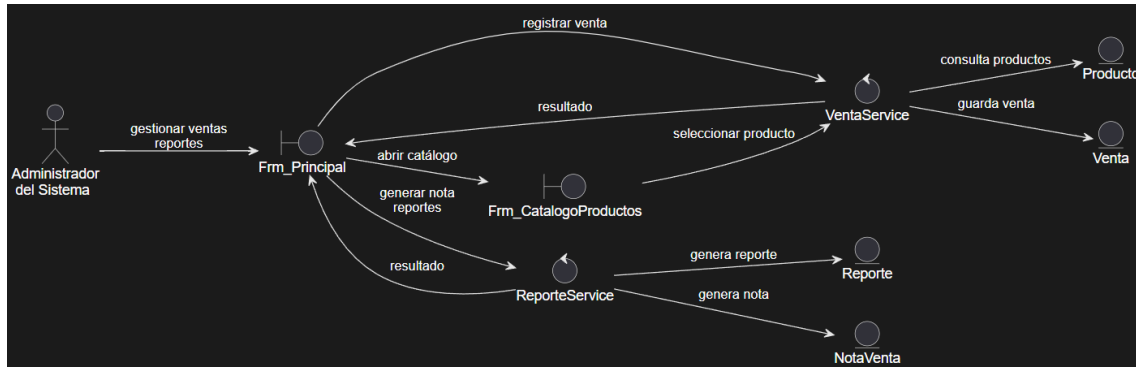


Ilustración 7 Puente PlantUML Sprint 3